

Seguridad y Buenas Prácticas

Mairin: El retrato roto

Ideathon 2026

BarrancoGroup

2026-08-30

Roles: Admin, Usuario y Auditor

El control de acceso del sistema se apoya en tres roles que delimitan, con alcances distintos, lo que cada actor puede hacer. El **administrador** concentra los privilegios de gestión: administrar usuarios, asignar o modificar roles y acceder a la funcionalidad administrativa. El **usuario** es el actor ordinario de la aplicación: usa la funcionalidad principal y crea datos asociados a su cuenta. El **auditor** es un rol orientado a la supervisión: consulta la información relevante y revisa los registros de operaciones, pero no modifica ni elimina aquello que audita. Esta última distinción materializa el principio de **separación de funciones**: quien supervisa no puede alterar lo que revisa.

En el código, los roles se expresan como una enumeración (ADMIN, USUARIO, AUDITOR) y se imponen con el decorador `@Roles ( . . . )` junto al guard `RolesGuard`, que compara el rol del token de acceso contra los roles exigidos por cada ruta. La matriz siguiente refleja únicamente lo que la aplicación implementa hoy:

Acción	Admin	Usuario	Auditor
Registrar sesión de juego	✓	✓	✗
Registrar un intento	✓	✓	✗
Consultar el progreso	✓	✓	✓
Registrarse / iniciar sesión	✓	✓	✓

El registro y el inicio de sesión son rutas públicas: cualquier visitante puede autenticarse para obtener un token con su rol. A partir de ahí, las rutas de progreso se protegen por rol: crear una sesión o registrar un intento está reservado al usuario y al administrador, mientras que la consulta del progreso también admite al auditor. Nótese que el auditor puede leer, pero no escribir: esa es la separación de funciones en acción.

Cabe aclarar que capacidades propias del rol administrador —como la gestión de usuarios, la revisión de registros de auditoría o la configuración del sistema— están previstas conceptualmente, pero aún no se exponen como endpoints; por eso no figuran en la matriz. El documento describe el sistema tal como está, sin atribuirle permisos que todavía no implementa.

Clean Code

El proyecto aplica los principios de código limpio mediante nombres descriptivos, separación de responsabilidades, funciones pequeñas, reutilización de lógica y manejo centralizado de errores. Los siguientes ejemplos provienen del código real del backend.

Responsabilidad única. PasswordService aísla por completo el cifrado y la verificación de contraseñas: ninguna otra clase conoce los detalles de scrypt, la sal aleatoria o la comparación en tiempo constante. Si mañana cambia el algoritmo de hashing, solo se toca este archivo.

```
@Injectable()
export class PasswordService {
  hash(plain: string): string {
    const salt = randomBytes(16).toString('hex');
    const hash = scryptSync(plain, salt, KEY_LENGTH).toString('hex');
    return `${salt}:${hash}`;
  }

  verify(plain: string, stored: string): boolean {
    const [salt, hash] = stored.split(':');
    const expected = Buffer.from(hash, 'hex');
    const actual = scryptSync(plain, salt, KEY_LENGTH);
    return expected.length === actual.length && timingSafeEqual(expected,
actual);
  }
}
```

Nombres descriptivos y funciones pequeñas. En JwtAuthGuard, cada método hace una sola cosa y su nombre lo declara: extractBearerToken extrae el token del encabezado; isPublic decide si una ruta está exenta de autenticación.

```
private isPublic(context: ExecutionContext): boolean {
  return this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
    context.getHandler(),
    context.getClass(),
```

```

    });
}

private extractBearerToken(request: Request): string | undefined {
    const [type, token] = request.headers.authorization?.split(' ') ?? [];
    return type === 'Bearer' ? token : undefined;
}

```

Reutilización de lógica (DRY). La emisión del token se concentra en un único método, `issueToken`, que comparten el registro y el inicio de sesión; no hay lógica de firma duplicada en dos lugares.

```

private issueToken(user: { id: string; username: string | null; role: Role }) {
    const payload: JwtPayload = {
        sub: user.id,
        username: user.username,
        role: user.role,
    };

    return { accessToken: this.jwt.sign(payload), role: user.role };
}

```

Validaciones

El backend rechaza los datos inválidos antes de que lleguen a la lógica de negocio o a la base de datos. La primera barrera son los DTO con decoradores de `class-validator`, aplicados por un `ValidationPipe` global que devuelve un error 400 ante cualquier incumplimiento. La segunda barrera son las restricciones propias de PostgreSQL: unicidad y claves foráneas.

Validación	Ejemplo real	Dónde
Campo requerido	<code>sessionToken</code> obligatorio en un intento	DTO
Formato	<code>sessionToken</code> debe ser un UUID válido	DTO

Validación	Ejemplo real	Dónde
Tipo de dato	success debe ser booleano	DTO
Longitud	username (3–32) y password (8–128)	DTO
Rol permitido	role ∈ ADMIN / USUARIO / AUDITOR	DTO
Valor único	username y sessionId no se repiten	PostgreSQL
Clave foránea	attempts.sessionId debe existir en game_sessions	PostgreSQL

El siguiente fragmento es el DTO que valida la creación de un intento:

```
export class CreateAttemptDto {
  @IsUUID(undefined, { message: 'sessionId debe ser un UUID válido' })
  sessionId: string;

  @IsString()
  @Length(1, 128)
  levelId: string;

  @IsBoolean()
  success: boolean;

  @IsOptional()
  @IsString()
  @Length(1, 128)
  endingId?: string;
}
```

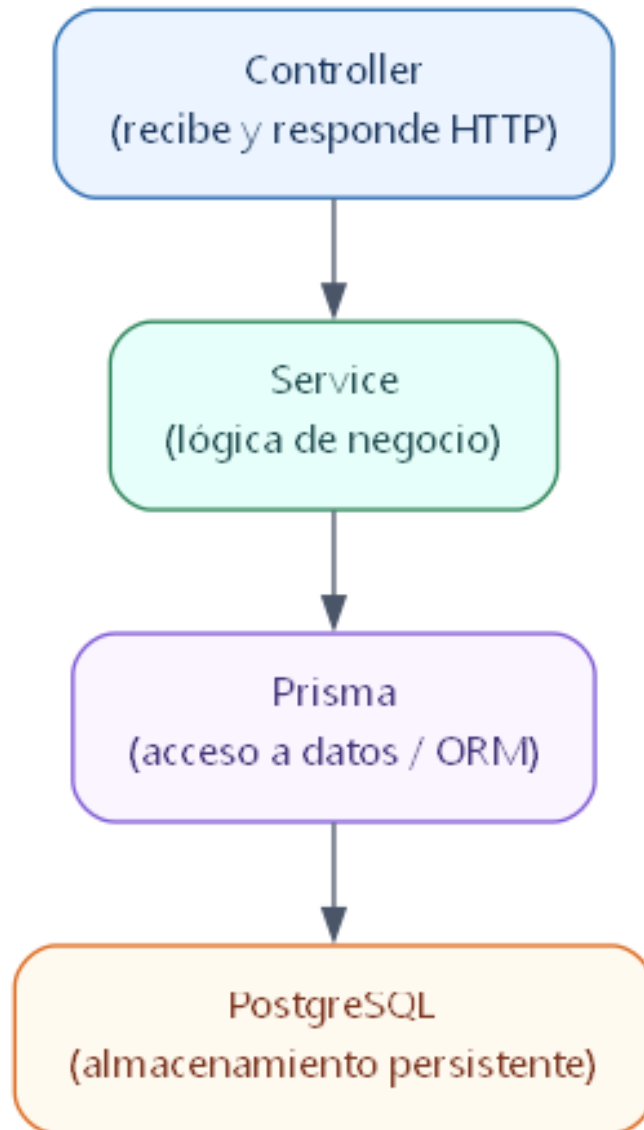
Y este otro valida el registro de un usuario, incluyendo el rol permitido:

```
export class RegisterDto {
  @IsString()
  @Length(3, 32)
  @Matches(/^[a-zA-Z0-9_.-]+$/, { message: 'username solo admite letras,
```

```
números y . _ -' })  
  username: string;  
  
  @IsString()  
  @Length(8, 128)  
  password: string;  
  
  @IsOptional()  
  @IsEnum(Role)  
  role?: Role;  
}
```

Organización

La estructura del backend sigue una separación por capas en la que cada nivel tiene una responsabilidad precisa y las dependencias apuntan siempre hacia adentro. El flujo de una petición recorre estas capas:



Controladores reciben la petición HTTP y devuelven la respuesta; no contienen reglas de negocio. **Servicios** concentran la lógica de la aplicación. **DTO** definen y validan los datos de entrada. **Guards** toman las decisiones de autenticación y autorización. **Prisma** centraliza el acceso a la base de datos a través del ORM, y **PostgreSQL** es el almacenamiento persistente.

La virtud de esta organización es que el acceso a datos no está disperso por toda la aplicación: toda consulta pasa por `PrismaService`, inyectado por dependencias, y los servicios lo consumen sin conocer los detalles de la conexión. Así, cambiar el ORM o la base de datos no toca la lógica de negocio, y la seguridad vive en una sola capa (los guards) en lugar de esparcirse en `if` por el código.

El resultado es un **modular monolith**: módulos de feature (auth, progress, prisma) con fronteras claras, validación en la frontera de entrada y preocupaciones transversales declarativas, que mantienen el sistema simple, testeable y ordenado.